Practical No 4.1

Singly Linked List (Call Log Management)

Name: Arjun Mankuskar

PRN:202401110038

Branch: CSE (AIML)

Batch: A2

```
//CODE
#include <iostream>
#include <string>
#include <limits>
using namespace std;

struct Node {
    string name;
    string number;
    Node* next;
};

Node* head = NULL;



void insertAtBegin(const string& name, const string& number) {
    Node* newNode = new Node();
    newNode->name = name;
    newNode->number = number;
    newNode->next = head;
    head = newNode;
    cout << "Contact added at beginning.\n";
}

void insertAtEnd(const string& name, const string& number) {
    Node* newNode = new Node();
    newNode->name = name;
    newNode->number = number;
    newNode->next = NULL;

    if (head == NULL)
```

```
        head = newNode;
    else {
        Node* temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    cout << "Contact added at end.\n";
}

void insertAtPos(const string& name, const string& number, int pos) {
    if (pos <= 0) {
        cout << "Invalid position.\n";
        return;
    }

    Node* newNode = new Node();
    newNode->name = name;
    newNode->number = number;

    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        cout << "Contact added at position 1.\n";
        return;
    }

    Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;
```

```
    if (temp == NULL) {
        cout << "Invalid position.\n";
        delete newNode;
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    cout << "Contact added at position " << pos << ".\n";
}

void deleteAtBegin() {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    Node* temp = head;
    head = head->next;
    delete temp;
    cout << "Contact deleted from beginning.\n";
}


void deleteAtEnd() {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    if (head->next == NULL) {
        delete head;
        head = NULL;
```

```
        cout << "Contact deleted from end.\n";
        return;
    }

    Node* temp = head;
    while (temp->next->next != NULL)
        temp = temp->next;

    delete temp->next;
    temp->next = NULL;
    cout << "Contact deleted from end.\n";
}



void deleteAtPos(int pos) {
    if (head == NULL) {
        cout << "List is empty.\n";
        return;
    }
    if (pos <= 0) {
        cout << "Invalid position.\n";
        return;
    }
    if (pos == 1) {
        Node* temp = head;
        head = head->next;
        delete temp;
        cout << "Contact deleted from position 1.\n";
        return;
    }
```

```
    Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) {
        cout << "Invalid position.\n";
        return;
    }

    Node* delNode = temp->next;
    temp->next = delNode->next;
    delete delNode;
    cout << "Contact deleted from position " << pos << ".\n";
}


void searchContact(const string& name) {
    Node* temp = head;
    while (temp != NULL) {
        if (temp->name == name) {
            cout << "Contact Found -> " << temp->name << " : " << temp->number << endl;
            return;
        }
        temp = temp->next;
    }
    cout << "Contact not found.\n";
}


void displayContacts() {
    if (head == NULL) {
```

```
        cout << "No contacts in call log.\n";
        return;
    }

    Node* temp = head;
    cout << "\nCall Log:\n";
    while (temp != NULL) {
        cout << "Name: " << temp->name << " | Number: " << temp->number << endl;
        temp = temp->next;
    }
}


void reverseList() {
    Node* prev = NULL;
    Node* curr = head;
    Node* next = NULL;

    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    head = prev;
    cout << "Call log reversed.\n";
}

int main() {
    int choice, pos;
    string name, number;
```

```
while (true) {
    cout << "\n--- Call Log Menu ---\n";
    cout << "1. Insert at Begin\n";
    cout << "2. Insert at End\n";
    cout << "3. Insert at Position\n";
    cout << "4. Delete at Begin\n";
    cout << "5. Delete at End\n";
    cout << "6. Delete at Position\n";
    cout << "7. Search Contact\n";
    cout << "8. Display Call Log\n";
    cout << "9. Reverse Call Log\n";
    cout << "10. Exit\n";
    cout << "Enter choice: ";
    if (!(cin >> choice)) {
        cout << "Invalid input. Please enter a number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        continue;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n'); // robustly clear rest of line

    switch (choice) {
    case 1:
        cout << "Enter full name: ";
        getline(cin >> ws, name);
        cout << "Enter number: ";
        getline(cin >> ws, number);
        insertAtBegin(name, number);
        break;
```

```
case 2:
    cout << "Enter full name: ";
    getline(cin >> ws, name);
    cout << "Enter number: ";
    getline(cin >> ws, number);
    insertAtEnd(name, number);
    break;

case 3:
    cout << "Enter full name: ";
    getline(cin >> ws, name);
    cout << "Enter number: ";
    getline(cin >> ws, number);
    cout << "Enter position: ";
    if (!(cin >> pos)) {
        cout << "Invalid position input.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    insertAtPos(name, number, pos);
    break;

case 4:
    deleteAtBegin();
    break;

case 5:
    deleteAtEnd();
    break;
```

```
case 6:
    cout << "Enter position to delete: ";
    if (!(cin >> pos)) {
        cout << "Invalid position input.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        break;
    }
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    deleteAtPos(pos);
    break;

case 7:
    cout << "Enter full name to search: ";
    getline(cin >> ws, name);
    searchContact(name);
    break;

case 8:
    displayContacts();
    break;

case 9:
    reverseList();
    break;

case 10:
    cout << "Exiting program.\n";
    // free all nodes before exit
    while (head != NULL) {
```

```
            Node* tmp = head;
            head = head->next;
            delete tmp;
        }
        return 0;

    default:
        cout << "Invalid choice.\n";
    }
  }
  return 0;
}

/*

--- Call Log Menu ---
1. Insert at Begin
2. Insert at End
3. Insert at Position
4. Delete at Begin
5. Delete at End
6. Delete at Position
7. Search Contact
8. Display Call Log
9. Reverse Call Log
10. Exit
Enter choice: 1
Enter full name: a
Enter number: 741
Contact added at beginning.
```

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 1

Enter full name: k

Enter number: 852

Contact added at beginning.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 2

Enter full name: l

Enter number: 963

Contact added at end.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 2

Enter full name: j

Enter number: 789

Contact added at end.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 3

Enter full name: o753

Enter number: 741

Enter position: 5

Contact added at position 5.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 8

Call Log:

Name: k | Number: 852

Name: a | Number: 741

Name: l | Number: 963

Name: j | Number: 789

Name: o753 | Number: 741

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 4

Contact deleted from beginning.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 8

Call Log:

Name: a | Number: 741

Name: l | Number: 963

Name: j | Number: 789

Name: o753 | Number: 741

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 5

Contact deleted from end.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 8

Call Log:

Name: a | Number: 741

Name: l | Number: 963

Name: j | Number: 789

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 7

Enter full name to search: a

Contact Found -> a : 741

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 9

Call log reversed.

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 8

Call Log:

Name: j | Number: 789

Name: l | Number: 963

Name: a | Number: 741

--- Call Log Menu ---

1. Insert at Begin

2. Insert at End

3. Insert at Position

4. Delete at Begin

5. Delete at End

6. Delete at Position

7. Search Contact

8. Display Call Log

9. Reverse Call Log

10. Exit

Enter choice: 10

Exiting program.

Process exited after 76.3 seconds with return value 0

Press any key to continue . . .

*/